Homework 5: Non-CFLs and Turing Machines

Will Griffin

March 20, 2026

1. Non-closure properties of CFLs

(a) Use the languages

$$A = \{\mathtt{a}^m\mathtt{b}^n\mathtt{c}^n \mid m, n \geq 0\}$$
$$B = \{\mathtt{a}^n\mathtt{b}^n\mathtt{c}^m \mid m, n \geq 0\}$$

to prove that the class of context-free languages is *not* closed under intersection.

To prove that the class of context-free languages is not closed under intersection, let's take the intersection of A and B.

$A \cap B = \{\mathtt{a}^n\mathtt{b}^n\mathtt{c}^n \mid n \geq 0\}$. This is because, when taking the intersection, if there are the same number of Bs and Cs in A, there must be an equal number of As in B since there are the same number of As and Bs in B. Therefore, the amount of each letter is equal in $A \cap B$.

However, $\{\mathtt{a}^n\mathtt{b}^n\mathtt{c}^n \mid n \geq 0\}$ is one of the non-context-free languages we discussed. Since the intersection of two context-free languages led to a non-context-free language result, the class of context-free language is not closed under intersection.

(b) Use Problem 1a and DeMorgan's law to prove that the class of context-free languages is *not* closed under complementation.

Assume for the sake of contradiction that the class of context-free languages is closed under complementation.

Therefore, $\overline{A}$ and $\overline{B}$ are both context-free. Since the class of context-free languages is closed under union, $\overline{A} \cup \overline{B}$ is also context-free.

Taking the complement again, $\overline{\overline{A} \cup \overline{B}}$ is still context-free.

DeMorgan's law states that for two languages L_1 and L_2, $\overline{L_1 \cup L_2} = \overline{L_1} \cap \overline{L_2}$. Taking the complement of each language, we get $\overline{\overline{L_1} \cup \overline{L_2}} = \overline{\overline{L_1}} \cap \overline{\overline{L_2}} = L_1 \cap L_2$.

Therefore, $\overline{\overline{A} \cup \overline{B}} = A \cap B$. However, we proved in Problem 1a that $A \cap B$ is non-context-free. This brings a contradiction.

Therefore, the class of context-free languages is not closed under complementation.

2. There and back again

Imagine a robot turtle that you can give instructions `u` (go up 1 cm), `d` (go down 1 cm), `l` (go left 1 cm), `r` (go right 1 cm). A program is a string of instructions.

Let C be the set of programs that make the turtle return to the starting point. For example, `uurdrdll` is in C.

(a) Prove that C is not context-free.

Assume for the sake of contradiction that C is context-free.

Define a regular language R to be $R = \texttt{u}^*\texttt{d}^*\texttt{l}^*\texttt{r}^*$. Intersect C with R. Since context-free languages are closed under intersection with regular languages, $C \cap R$ is context-free.

Then, define a string homomorphism ϕ such that $\phi(w) = \phi(w_1)\phi(w_2)\cdots\phi(w_n)$ and for each letter,

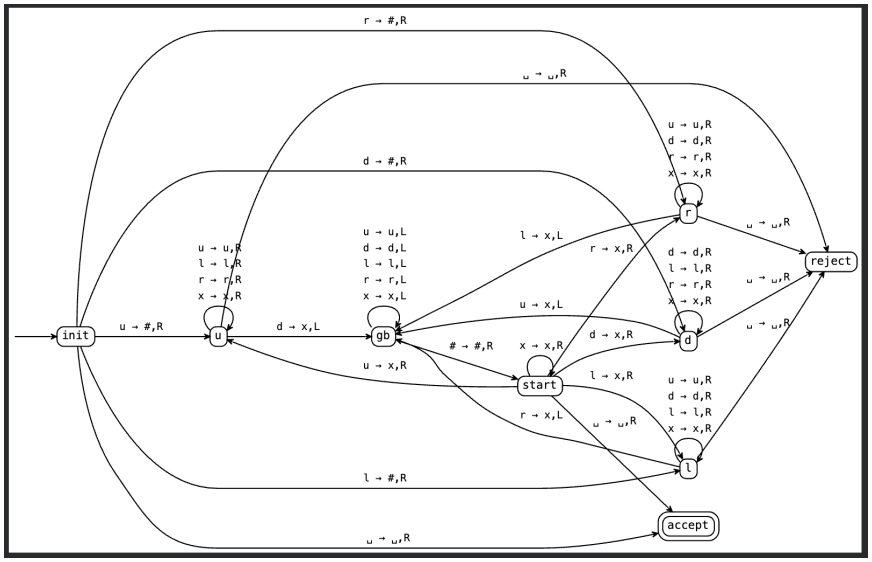
$$\phi(v) = \begin{cases} u & \text{if } v = u \\ d & \text{if } v = d \\ l & \text{if } v = l \\ \varepsilon & \text{if } v = r \end{cases}$$

Apply the string homomorphism to the intersection, that is, $\phi(C \cap R)$. Since context-free languages are closed under string homomorphisms, $\phi(C \cap R)$ is context-free.

However, since $C \cap R$ is equal to $\{\texttt{u}^n\texttt{d}^n\texttt{l}^n\texttt{r}^n \mid n \geq 0\}$, and the string homomorphism ϕ removes all `r`s, $\phi(C \cap R)$ is equal to $\{\texttt{u}^n\texttt{d}^n\texttt{l}^n \mid n \geq 0\}$.

This language is identical to $\{\texttt{a}^n\texttt{b}^n\texttt{c}^n \mid n \geq 0\}$, which is a well-known non-context-free language. This is a contradiction, since $\phi(C \cap R)$ was context-free by closure properties. Therefore, C is a not context-free.

(b) Write a **formal description** of a Turing machine that decides C.



The CSV file for the Turing machine can be found at https://gist.github.com/whgriff05/f39eafeecf4e97040af2714cfacb2af3.

3. Turing closure properties

(a) Write an **implementation-level** description of a Turing machine S that, on input $v \in \Sigma^*$, decides whether $v = \text{STRETCH}(u)$ for some u. Moreover, if S accepts v, then when it halts, the contents of the tape should be u. For example, if the input is `001100`, S should accept and the final contents of the tape should be `010`. But if the input is `001101`, S should reject.

$S =$ "On input string v,

1. Start at the left end of the tape. If the first symbol is a blank symbol, *accept*.
2. Read the first symbol (to be referred to as the symbol being processed), change it to a blank symbol, then move the head to the right. If the next symbol is not the same as the symbol being processed, *reject*.
3. Otherwise, if it is the same, change it to a blank symbol, and move left to either the left end of the tape or the previous non-blank symbol. If on the left end, do not move. If on the previous non-blank symbol, move to the right once.
4. Write the symbol currently being processed.
5. Move to the right to the next non-blank symbol. If there are no more non-blank symbols, *accept*. Otherwise, upon reaching the next non-blank symbol, go back to step 2."

(b) Prove that if L is a Turing-decidable language over Σ, then STRETCH(L) is also Turing-decidable. You should let M be a Turing machine that decides L, then use your answer to 3a to give an **implementation-level** description of a Turing machine that decides STRETCH(L). One of the lines of your description can be "Simulate M".

If L is a Turing-decidable language, that means some Turing machine M decides L. Let another Turing machine M' decide STRETCH(L).

$M' =$ "On input string $v \in$ STRETCH(L),

1. Simulate S (from Problem 3a) on v. If S accepts, continue. Otherwise, if S rejects, *reject*.
2. Move the tape left to the left end.
3. Simulate M.

Since the Turing machine S from Problem 3a only accepts strings of the form STRETCH(v) and converts the tape to v, any accepted stretched string will be converted to its pre-stretched form starting at the left end of the tape when S accepts. Then, starting back from the left end, M will decide on v. Since S transmutes a stretched string into its original form, and M can decide on the original string, STRETCH(L) is Turing-decidable.

(c) Prove that if L is a Turing-recognizable language, then STRETCH(L) is also Turing recognizable.

A Turing-recognizable language is defined as some Turing machine that halts and accepts strings in the language and either halts and rejects or loops on strings not in the language. Therefore, if L is Turing-decidable, that is, some Turing machine halts and accepts strings in the language and halts and rejects strings not in the language, L is also Turing-recognizable, as it follows the same behavior for strings in the language and one of the accepted behaviors for strings not in the language.

Since STRETCH(L) is Turing-decidable (Problem 3b), therefore, STRETCH(L) is also Turing-recognizable.